Importar librerías

In [65]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

import statsmodels.api as sm

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report, confusion_matrix, roc_auc_score,

%matplotlib inline
```

Conseguir data

In [66]:
```
ad_data = pd.read_csv('advertising.csv')
```

In [67]:
```
ad_data.head()
```

Out[67]:

	Daily Time Spent on Site	Age	Area Income	Daily Internet Usage	Ad Topic Line	City	Male	Country	Timestamp
0	68.95	35	61833.90	256.09	Cloned 5thgeneration orchestration	Wrightburgh	0	Tunisia	2016-03-27 00:53:11
1	80.23	31	68441.85	193.77	Monitored national standardization	West Jodi	1	Nauru	2016-04-04 01:39:02
2	69.47	26	59785.94	236.50	Organic bottom-line service-desk	Davidton	0	San Marino	2016-03-13 20:35:42
3	74.15	29	54806.18	245.89	Triple-buffered reciprocal time-frame	West Terrifurt	1	Italy	2016-01-10 02:31:19
4	68.37	35	73889.99	225.58	Robust logistical utilization	South Manuel	0	Iceland	2016-06-03 03:36:18

In [68]:
```
ad_data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1000 entries, 0 to 999
Data columns (total 10 columns):
 #   Column                    Non-Null Count  Dtype
---  ------                    --------------  -----
 0   Daily Time Spent on Site  1000 non-null   float64
 1   Age                       1000 non-null   int64
 2   Area Income               1000 non-null   float64
 3   Daily Internet Usage      1000 non-null   float64
 4   Ad Topic Line             1000 non-null   object
 5   City                      1000 non-null   object
 6   Male                      1000 non-null   int64
 7   Country                   1000 non-null   object
 8   Timestamp                 1000 non-null   object
 9   Clicked on Ad             1000 non-null   int64
dtypes: float64(3), int64(3), object(4)
memory usage: 78.3+ KB
```

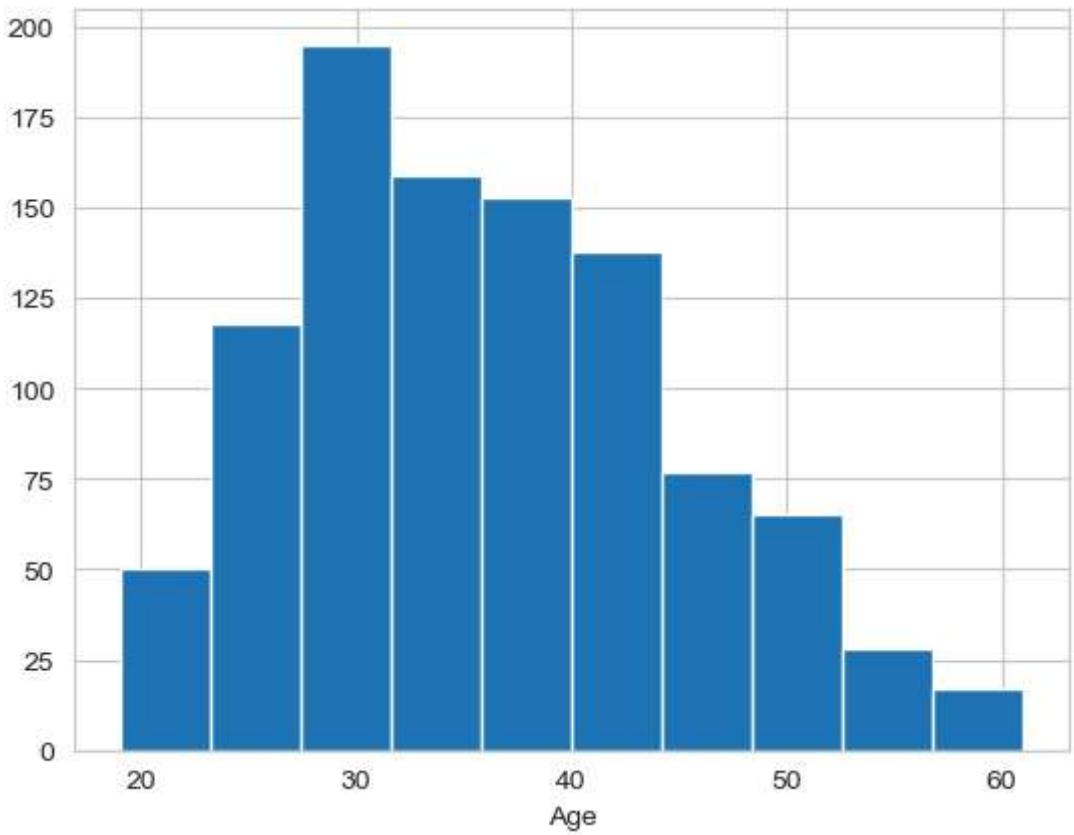
In [69]:
```
ad_data['Clicked on Ad'].value_counts()
```

Out[69]:
```
Clicked on Ad
0    500
1    500
Name: count, dtype: int64
```

Análisis Exploratorio

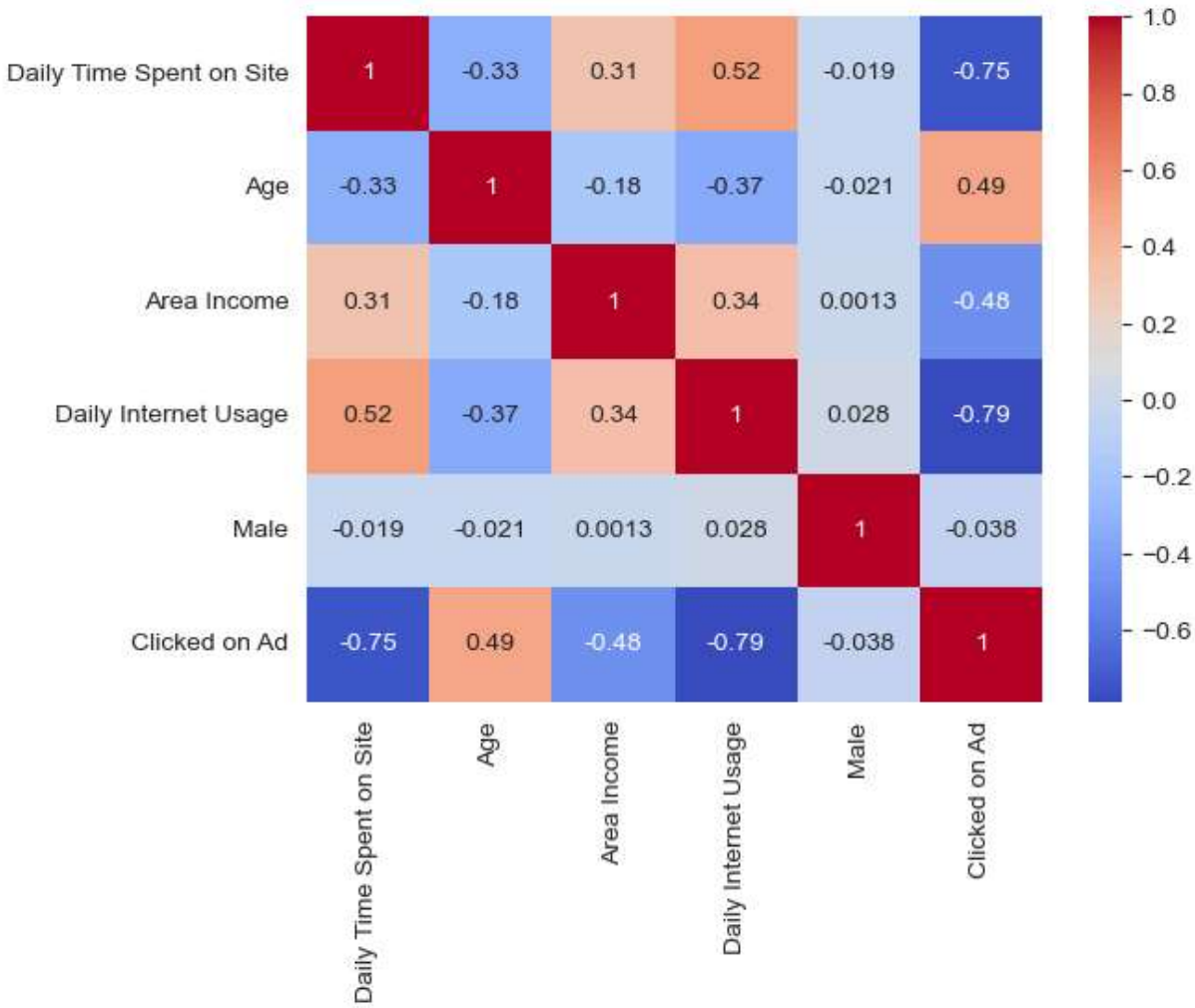
In [70]:
```
sns.set_style('whitegrid')
ad_data['Age'].hist(bins=10)
plt.xlabel('Age')
```

Out[70]:
```
Text(0.5, 0, 'Age')
```



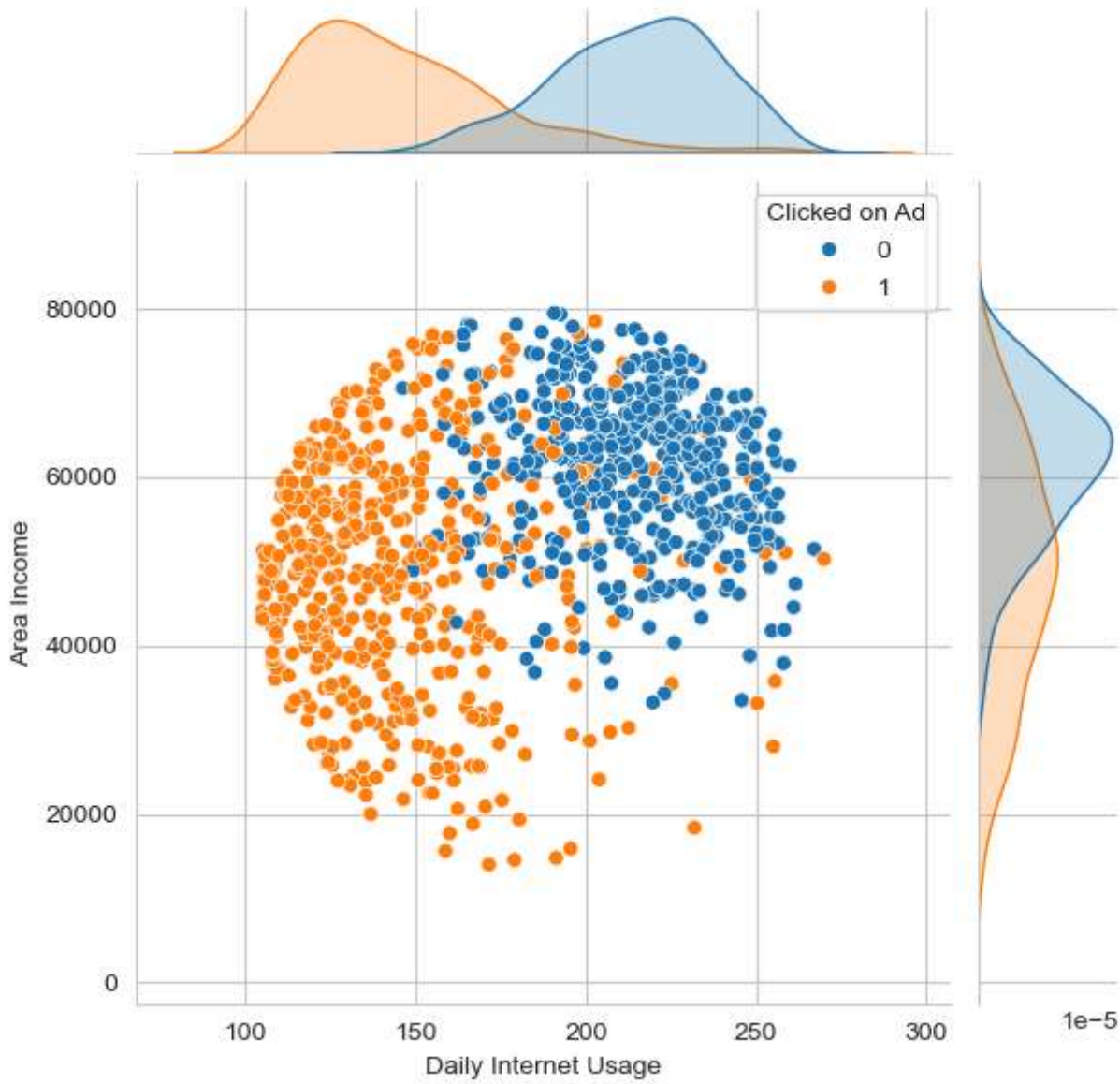
In [71]:
```
sns.heatmap(ad_data.corr(numeric_only=True), annot=True, cmap='coolwarm')
```

Out[71]: <Axes: >



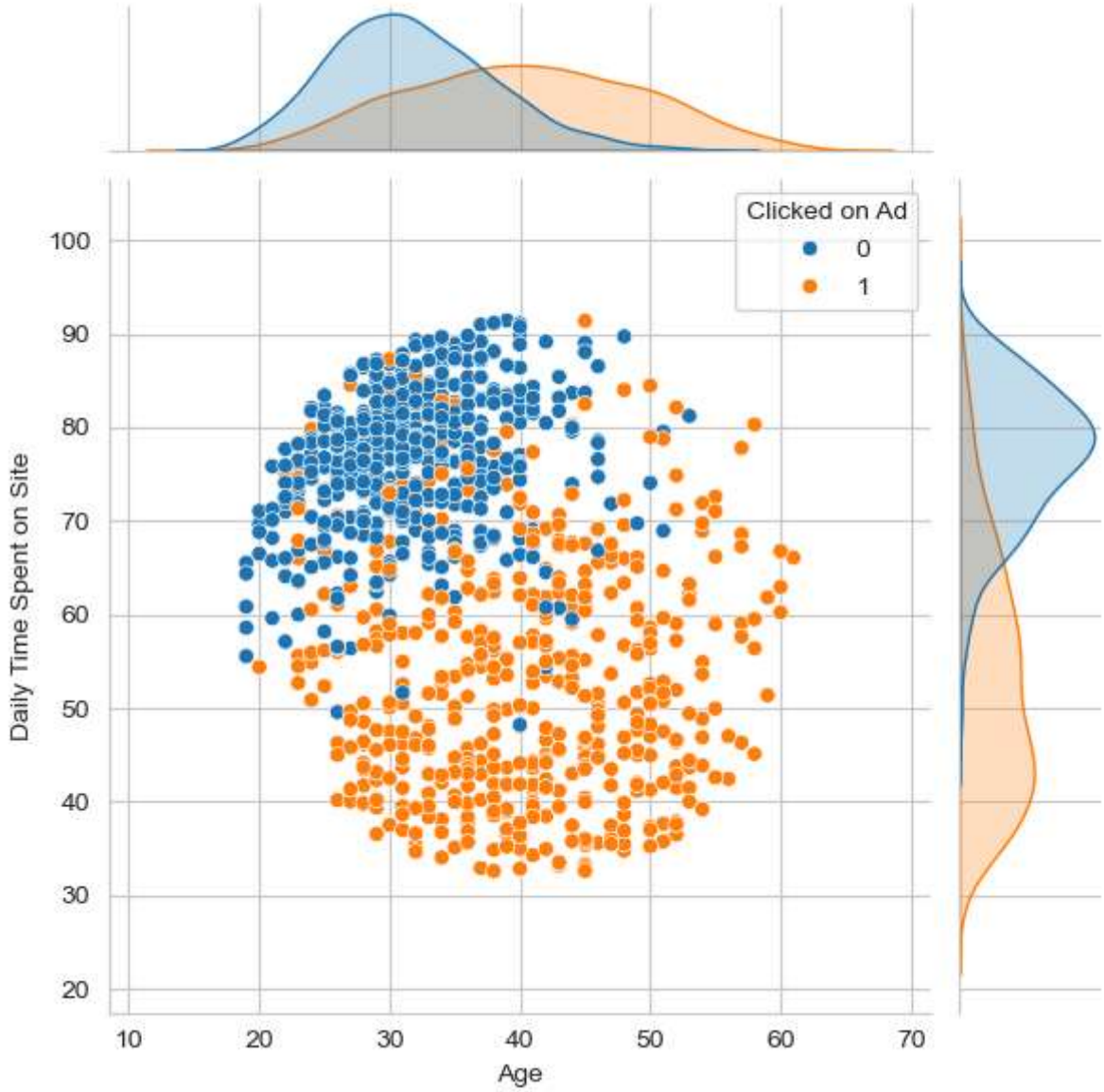
In [72]:
```
sns.jointplot(
    x='Daily Internet Usage',
    y='Area Income',
    data=ad_data,
    hue='Clicked on Ad'
)
```

Out[72]: <seaborn.axisgrid.JointGrid at 0x212b5d57df0>



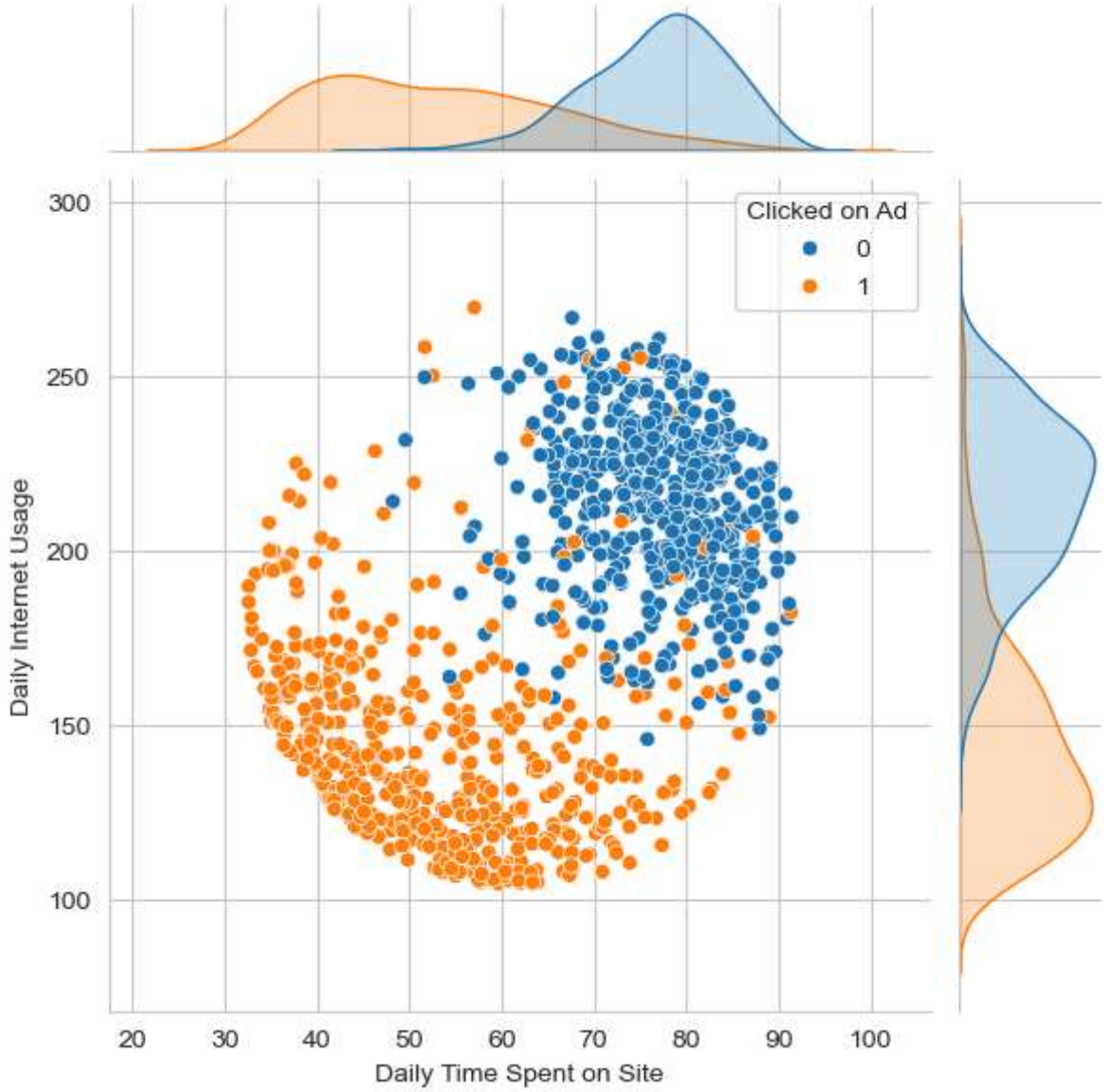
In [73]:
```
sns.jointplot(
    x='Age',
    y='Daily Time Spent on Site',
    data=ad_data,
    hue='Clicked on Ad',
    kind='scatter'
)
```

Out[73]: <seaborn.axisgrid.JointGrid at 0x212b5d57ac0>



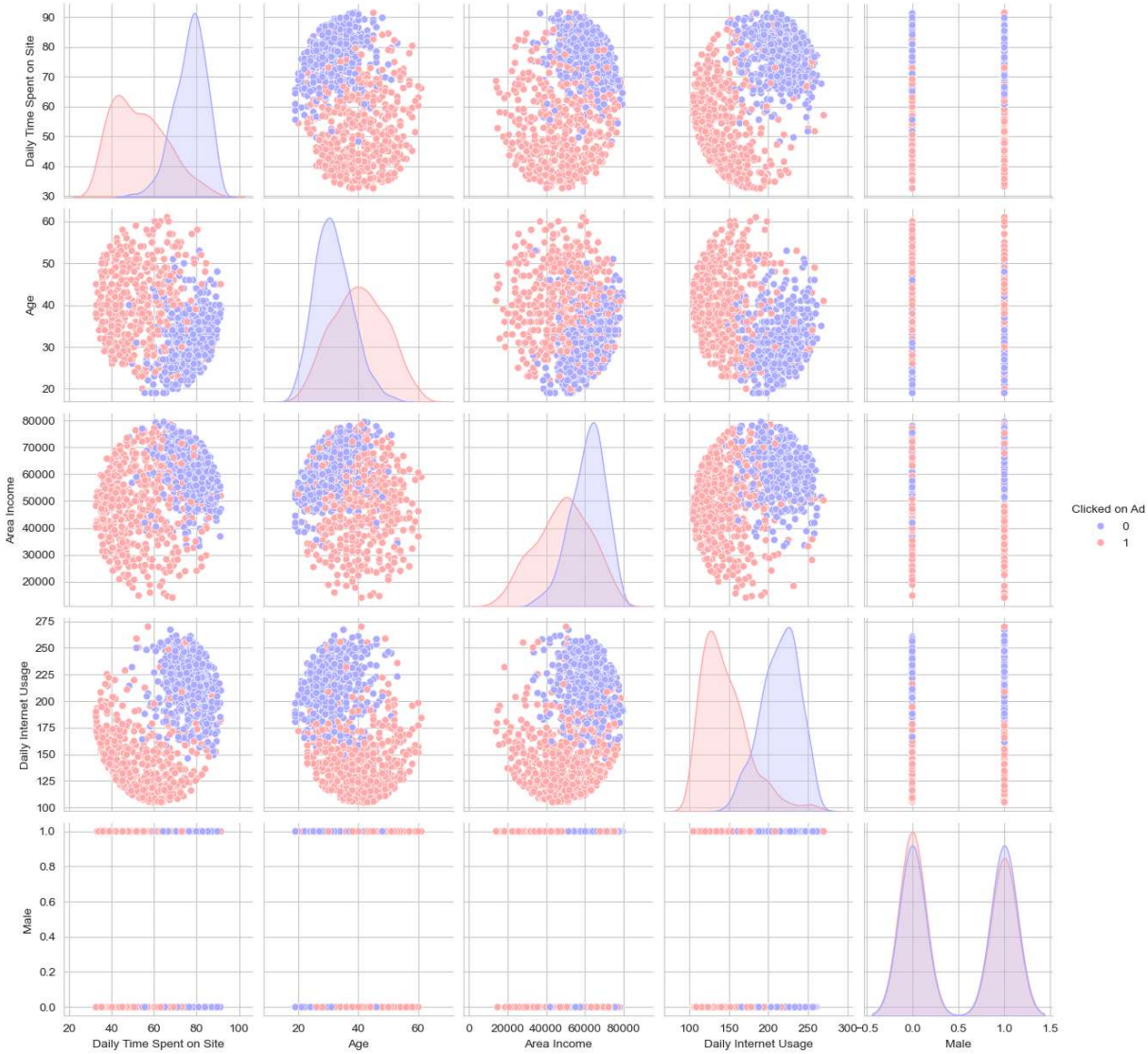
In [74]:
```
sns.jointplot(
    x='Daily Time Spent on Site',
    y='Daily Internet Usage',
    data=ad_data,
    hue='Clicked on Ad'
)
```

Out[74]: <seaborn.axisgrid.JointGrid at 0x212b5bcb250>



In [75]:
```
sns.pairplot(
    ad_data,
    hue='Clicked on Ad',
    palette='bwr'
)
```

Out[75]: <seaborn.axisgrid.PairGrid at 0x212be02ead0>



Logistic Regression

In [76]:
```
from sklearn.model_selection import train_test_split
```

In [77]:
```
X = ad_data[['Daily Time Spent on Site',
             'Age',
             'Area Income',
             'Daily Internet Usage',
             'Male']]

y = ad_data['Clicked on Ad']
```

In [78]:
```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.33, random_state=42
)
```

In [79]:
```
from sklearn.linear_model import LogisticRegression
```

In [80]:
```
logmodel = LogisticRegression()
logmodel.fit(X_train, y_train)
```

```
C:\Users\Diego\anaconda3\Lib\site-packages\sklearn\linear_model\_logistic.py:473: Co
nvergenceWarning: lbfgs failed to converge after 100 iteration(s) (status=1):
STOP: TOTAL NO. OF ITERATIONS REACHED LIMIT

Increase the number of iterations to improve the convergence (max_iter=100).
You might also want to scale the data as shown in:
    https://scikit-learn.org/stable/modules/preprocessing.html
Please also refer to the documentation for alternative solver options:
    https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression
  n_iter_i = _check_optimize_result(
```

Out[80]:

```
▾ LogisticRegression
▶ Parameters
```

Predicciones y Evaluaciones

In [81]:
```
predictions = logmodel.predict(X_test)
```

In [82]:
```
from sklearn.metrics import classification_report
```

In [83]:
```
print(classification_report(y_test, predictions))
```

```
              precision    recall  f1-score   support

           0       0.86      0.96      0.91       162
           1       0.95      0.85      0.90       168

    accuracy                           0.90       330
   macro avg       0.91      0.90      0.90       330
weighted avg       0.91      0.90      0.90       330
```

In [84]:
```
from sklearn.metrics import confusion_matrix

print(confusion_matrix(y_test, predictions))
```

```
[[155   7]
 [ 25 143]]
```

In [85]:
```
print(logmodel.intercept_)
```

```
[0.60370368]
```

In [86]:
```
coeff_df = pd.DataFrame(logmodel.coef_[0], X.columns, columns=['Coefficient'])
coeff_df
```

Out[86]:

	Coefficient
Daily Time Spent on Site	-0.037544
Age	0.272808
Area Income	-0.000042
Daily Internet Usage	-0.030404
Male	0.161735

In [87]:
```
coeff_df = pd.DataFrame(logmodel.coef_[0], X.columns, columns=['Coefficient'])
coeff_df
```

Out[87]:

	Coefficient
Daily Time Spent on Site	-0.037544
Age	0.272808
Area Income	-0.000042
Daily Internet Usage	-0.030404
Male	0.161735

In [88]:
```
logmodel2 = sm.Logit(y_train, X_train).fit()
print(logmodel2.summary())
```

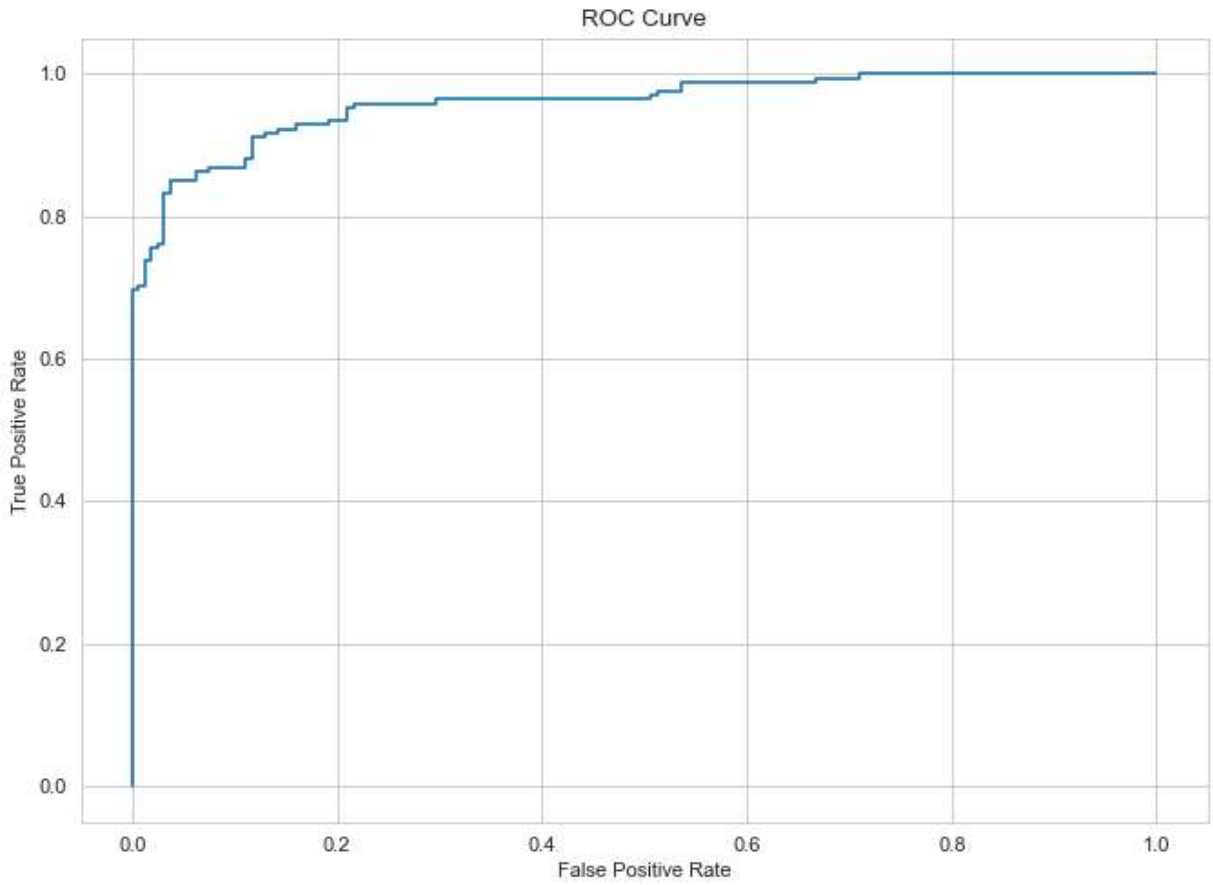
```
Optimization terminated successfully.
         Current function value: 0.253610
         Iterations 8
                           Logit Regression Results
==============================================================================
Dep. Variable:          Clicked on Ad   No. Observations:                  670
Model:                          Logit   Df Residuals:                      665
Method:                           MLE   Df Model:                            4
Date:                Mon, 20 Apr 2026   Pseudo R-squ.:                  0.6341
Time:                        20:19:57   Log-Likelihood:                -169.92
converged:                       True   LL-Null:                       -464.38
Covariance Type:            nonrobust   LLR p-value:                3.861e-126
==================================================================================
========
                              coef    std err          z      P>|z|      [0.025
0.975]
----------------------------------------------------------------------------------
--------
Daily Time Spent on Site    -0.0533      0.010     -5.433      0.000      -0.073
-0.034
Age                          0.2596      0.021     12.647      0.000       0.219
0.300
Area Income              -1.707e-05   9.86e-06     -1.731      0.083   -3.64e-05
2.25e-06
Daily Internet Usage        -0.0276      0.004     -7.115      0.000      -0.035
-0.020
Male                         0.3019      0.287      1.051      0.293      -0.261
0.865
==================================================================================
========
```

In [89]:
```
def plot_roc_curve(y_true, y_prob):
    fpr, tpr, thresholds = roc_curve(y_true, y_prob)
    plt.plot(fpr, tpr)
    plt.xlabel('False Positive Rate')
    plt.ylabel('True Positive Rate')
    plt.title('ROC Curve')
    plt.show()
```

In [90]:
```
roc_curve(y_test, logmodel.predict_proba(X_test)[:,1])[1]
```

Out[90]:
```
array([0.        , 0.00595238, 0.69642857, 0.69642857, 0.70238095,
       0.70238095, 0.73809524, 0.73809524, 0.75595238, 0.75595238,
       0.76190476, 0.76190476, 0.83333333, 0.83333333, 0.85119048,
       0.85119048, 0.86309524, 0.86309524, 0.86904762, 0.86904762,
       0.88095238, 0.88095238, 0.91071429, 0.91071429, 0.91666667,
       0.91666667, 0.92261905, 0.92261905, 0.92857143, 0.92857143,
       0.93452381, 0.93452381, 0.95238095, 0.95238095, 0.95833333,
       0.95833333, 0.96428571, 0.96428571, 0.9702381 , 0.9702381 ,
       0.97619048, 0.97619048, 0.98809524, 0.98809524, 0.99404762,
       0.99404762, 1.        , 1.        ])
```

In [92]:
```
plt.figure(figsize=(10,7), dpi=80)
plot_roc_curve(y_test, logmodel.predict_proba(X_test)[:,1])
```



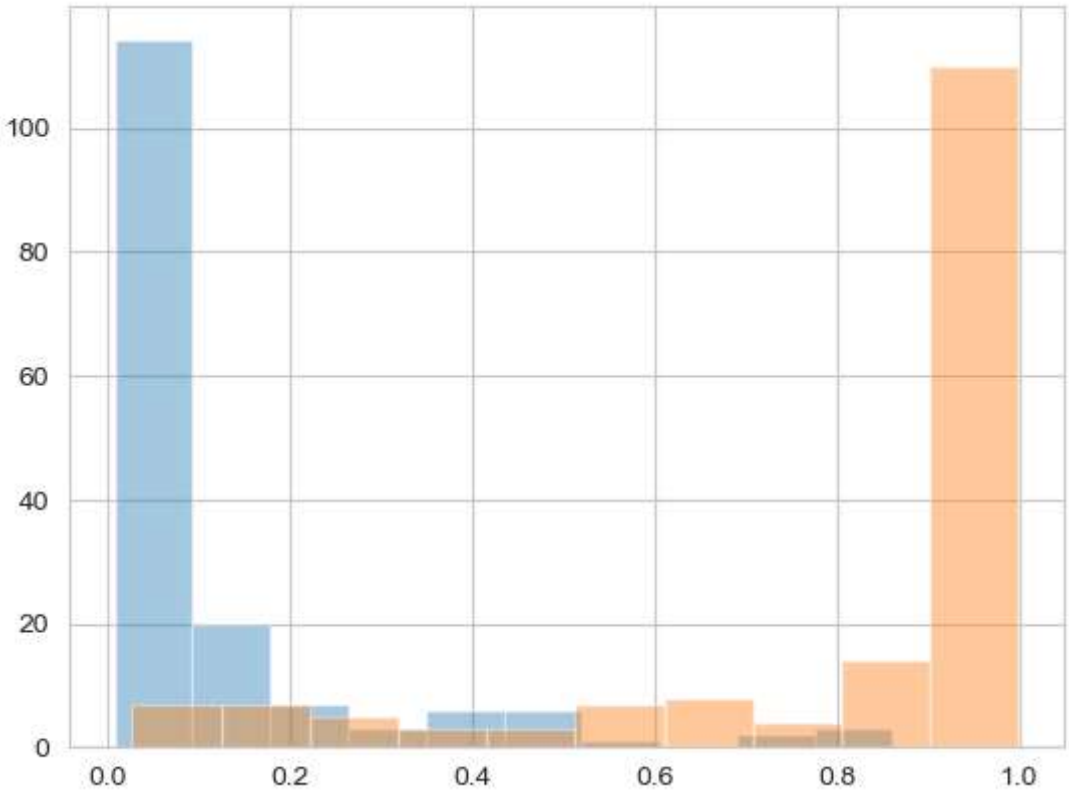
In [93]:
```
roc_auc_score(y_test, logmodel.predict_proba(X_test)[:,1])
```

Out[93]:
```
0.9590314520870077
```

In [94]:
```
graph = pd.DataFrame({
    'clicked': y_test,
    'probability': logmodel.predict_proba(X_test)[:,1]
})

graph.groupby('clicked').probability.hist(alpha=0.4)
```

Out[94]:
```
clicked
0    Axes(0.125,0.11;0.775x0.77)
1    Axes(0.125,0.11;0.775x0.77)
Name: probability, dtype: object
```



In [ ]: